

Performance of State-of-the-Art Machine Learning Methods on Out-of-Distribution Data in Tabular Regression: A Survey

Bao Minh Tran - s224236373

2025 ADR Summer Project
Deakin University

February 27, 2026

Table of Content

1 Problem settings

- Problem formulation
- Distributional shifts
- Convex hull of data

2 Splitting methods

- Random and Distribution-based methods
- Single Hyperball
- Multiple Hyperballs
- K-Means Hyperballs
- Single Slab
- Semi-infinite Slab

3 Results

- Main findings
- Side experiment

4 Conclusion and Discussion

(True) Population Risk [2]

Given an input space or domain, denoted as $\mathcal{X}$, an approximation function, denoted as $\hat{f} : \mathcal{X} \mapsto \mathcal{Y} \subseteq \mathbb{R}$, and f is the true function representing the relationship $\mathcal{X} \mapsto \mathcal{Y}$. The models repeatedly update their parameters to [1] diminish the population risk.

$$\mathcal{R}_{\text{true}}(\hat{f}) \equiv \mathcal{R}_{\mathbf{x} \in \mathcal{X}}(\hat{f}) \stackrel{\text{def}}{=} \mathbb{E}_{\mathbf{x} \in \mathcal{X}} \left[\ell(\hat{f}(\mathbf{x}), f(\mathbf{x})) \right]$$

Empirical Risk [2] and Population Risk

Nonetheless, in real-world scenarios, it is almost impossible to capture all behaviours of $\mathcal{X}$ to train the models. Alternatively, the model $\hat{f}$ is trained on a **sub-domain** $\mathcal{X}^{\text{tr}} \subset \mathcal{X}$, and hence, the model $\hat{f}$ is trying to optimize the sample risk or empirical risk.

$$\mathcal{R}_{\text{emp}}(\hat{f}) \equiv \mathcal{R}_{\mathbf{x} \in \mathcal{X}^{\text{tr}}}(\hat{f}) \stackrel{\text{def}}{=} \mathbb{E}_{\mathbf{x} \in \mathcal{X}^{\text{tr}}} \left[\ell(\hat{f}(\mathbf{x}), f(\mathbf{x})) \right]$$

The model $\hat{f}$ is expected to [2] make the **Empirical Risk** $\mathcal{R}_{\text{emp}}$ approximates the **True Risk** $\mathcal{R}_{\text{true}}$. Formally, this is

$$\arg \min_{\hat{f}} \mathcal{R}_{\text{emp}}(\hat{f}) \approx \arg \min_{\hat{f}} \mathcal{R}_{\text{true}}(\hat{f})$$

Distributional shifts

Call $\mathbf{X}$ a finite set of data points and $\mathbf{y}$ is the corresponding set of outputs. $\mathbf{P}^{\text{tr}}$ is the distribution of train data sampled from $\mathbf{X}$, and $\mathbf{P}^{\text{te}}$ is the distribution of test data sampled from $\mathbf{X}$.

Definition 3. No shifts - In-Distribution (ID) [2]

- $\mathbf{P}^{\text{te}}(\mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

Definition 4. Covariate shifts [3]

- $\mathbf{P}^{\text{te}}(\mathbf{X}) \neq \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

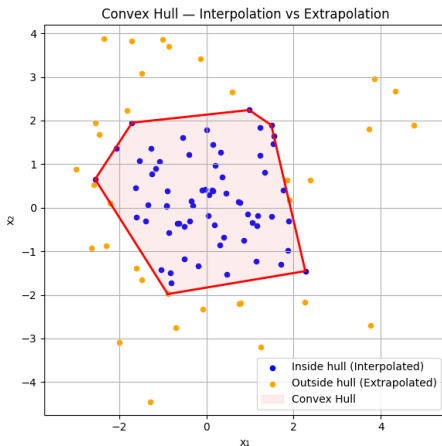
Definition 5. Concept/Semantic shifts [3]

- $\mathbf{P}^{\text{te}}(\mathbf{X}) = \mathbf{P}^{\text{tr}}(\mathbf{X})$
- $\mathbf{P}^{\text{te}}(\mathbf{y} \mid \mathbf{X}) \neq \mathbf{P}^{\text{tr}}(\mathbf{y} \mid \mathbf{X})$

Interpolation vs. Extrapolation with Convex Hull [4] [5]

Given a model $\hat{f}$ trained on a set of points $\mathcal{X}^{\text{tr}}$, the convex hull of $\mathcal{X}^{\text{tr}}$, denoted as $\text{Conv}(\mathcal{X}^{\text{tr}})$. For any point $\mathbf{x} \sim \mathcal{X}^{\text{te}}$ and $\mathcal{X}^{\text{te}} \cap \mathcal{X}^{\text{tr}} = \emptyset$:

- Interpolation occurs when $\mathbf{x} \in \text{Conv}(\mathcal{X}^{\text{tr}})$.
- Extrapolation occurs when $\mathbf{x} \notin \text{Conv}(\mathcal{X}^{\text{tr}})$.



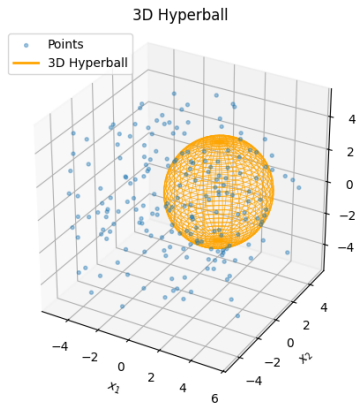
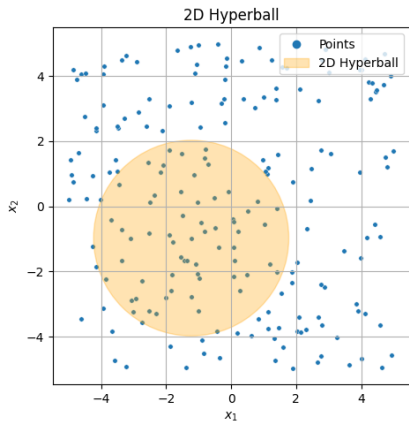
Random and Distribution-based methods

- Random method
- Covariate-based method
- Concept-based method (Metafeature-based method)

Single Hyperball

A Hyperball, or ball $\mathbf{B}_r(\mathbf{w})$ of radius r about $\mathbf{w} \in \mathbb{R}^n$ is defined as:

$$\mathbf{B}_r(\mathbf{w}) \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid d(\mathbf{x}, \mathbf{w}) \leq r\}$$



Multiple Hyperballs and KMeans Hyperballs

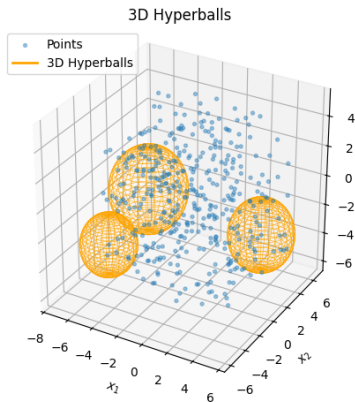
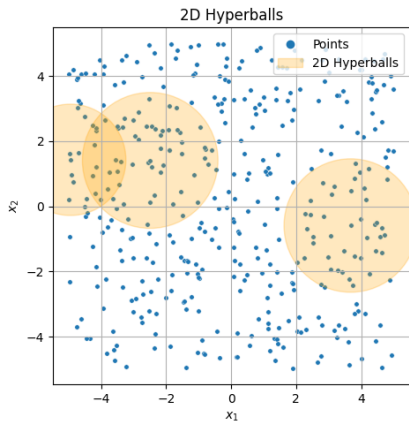
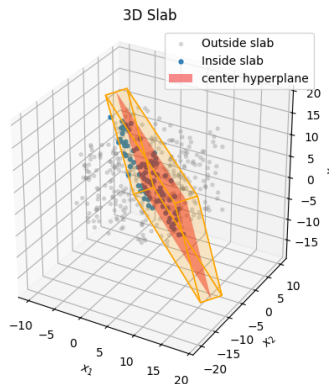
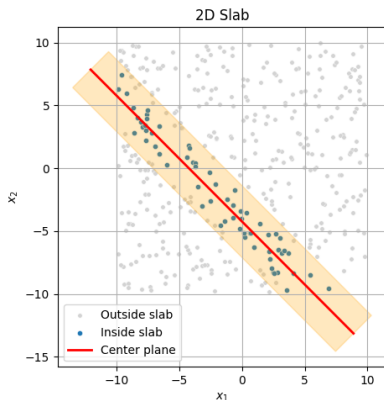


Figure: Multiple Hyperballs Visualization

Single Slab

Given a hyperplane $P = \{\mathbf{x} \mid \mathbf{n}^\top \mathbf{x} = b\} \subset \mathbb{R}^n$. A δ -slab defined by hyperplane P , is given by:

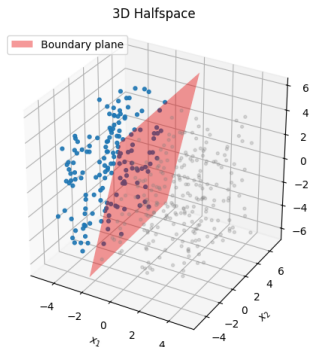
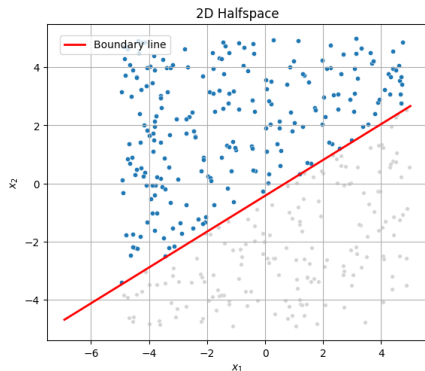
$$\text{Slab}_\delta(P) \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid d(\mathbf{x}, P) < \delta\} \subset \mathbb{R}^n, \quad \text{for } \delta \in \mathbb{R}$$



Semi-infinite Slab

Given a hyperplane $P = \{\mathbf{x} \mid \mathbf{n}^\top \mathbf{x} = b\} \subset \mathbb{R}^n$. A **semi-infinite slab** defined by hyperplane P , is given by:

$$\text{Slab}_\infty(P) \stackrel{\text{def}}{=} \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{n}^\top \mathbf{x} < b\}$$



Across-dataset split-agnostic performance of all models

Table 1: Across-dataset split-agnostic performance of all models. Lower mean ranks indicate better robustness across datasets regardless of split regimes.

Models	Mean-rank
LinearRegressor	8.967033
PolynomialRegressor	9.752747
KNNRegressor	9.631868
SVMRegressor	10.329670
DTRRegressor	9.857143
RFRRegressor	6.543956
GBRegressor	4.956044
ABRegressor	6.565934
XGBRegressor	5.219780
LightGBMRegressor	4.675824
RealMLPRegressor	4.351648
ResnetRegressor	5.203297
FTTransformerRegressor	4.945055
Friedman χ^2	782.8004
Significance	*
Significance codes: * $p \lll 0.001$	

Across-dataset split-wise performance of all models

Table 2: Across-dataset split-wise performance of all models. Lower mean ranks indicate better robustness across datasets w.r.t split regimes.

Models	#1	#2	#3	#4	#5	#6	#7	#8
LinearRegressor	11.08	8.96	9.38	8.81	8.92	9.00	9.00	8.69
PolynomialRegressor	10.15	8.65	9.62	10.46	8.35	9.42	11.00	10.77
KNNRegressor	7.42	10.12	9.42	9.08	10.31	9.73	9.38	9.38
SVMRegressor	12.31	10.04	10.92	10.23	9.96	10.31	10.69	10.15
DTRRegressor	9.73	10.69	10.31	8.81	10.69	10.00	9.23	9.27
RFRegressor	6.73	6.92	6.88	6.08	6.88	6.81	6.19	6.04
GBRegressor	5.04	5.12	4.81	4.69	5.35	5.38	4.31	5.04
ABRegressor	6.19	7.12	6.73	6.23	6.85	6.42	6.08	6.54
XGBRegressor	5.00	5.73	5.00	5.27	5.69	5.27	4.58	5.00
LightGBMRegressor	2.92	4.85	4.54	5.00	4.23	5.04	4.35	4.73
RealMLPRegressor	2.81	3.96	3.46	5.23	4.08	4.04	4.88	4.81
ResnetRegressor	4.85	3.35	4.81	6.58	4.00	4.96	6.50	6.23
FTTransformerRegressor	6.77	5.50	5.12	4.54	5.69	4.62	4.81	4.35
Friedman χ^2	188.3	126.5	138.1	96.7	120.5	112.6	129.4	110.5
Significance	*	*	*	*	*	*	*	*
<i>Significance codes: * $p \lll 0.001$</i>								

Side result 1 - Split-agnostic table

Table 3: Across-dataset split-agnostic performance of all models. Lower mean ranks indicate better robustness across datasets regardless of split regimes.

Models	Mean-rank
LinearRegressor	8.692308
PolynomialRegressor	9.492308
KNNRegressor	9.753846
SVMRegressor	10.346154
DTRegressor	9.950000
RFRegressor	6.346154
GBRegressor	4.884615
ABRegressor	6.196154
XGBRegressor	5.469231
LightGBMRegressor	5.015385
RealMLPRegressor	4.469231
ResnetRegressor	5.200000
FTTransformerRegressor	5.184615
Friedman χ^2	782.8004
Significance	*
<i>Significance codes: * $p \lll 0.001$</i>	

Side result 2 - Split-wise table

Table 4: Across-dataset split-wise performance of all models. Lower mean ranks indicate better robustness across datasets w.r.t split regimes.

Models	#4	#5	#6	#7	#8
LinearRegressor	8.96	8.62	8.81	8.85	8.23
PolynomialRegressor	9.15	9.73	9.15	9.23	10.19
KNNRegressor	9.85	9.35	9.46	10.12	10.00
SVMRegressor	10.31	10.19	9.88	10.92	10.42
DTRRegressor	10.38	9.69	9.81	10.65	9.21
RFRRegressor	6.38	6.23	6.88	6.27	5.96
GBRegressor	4.88	4.92	5.19	4.85	4.58
ABRegressor	6.35	6.23	6.62	6.27	5.52
XGBRegressor	5.50	5.50	5.77	5.42	5.15
LightGBMRegressor	5.23	5.23	4.88	4.77	4.96
RealMLPRegressor	4.15	4.73	4.42	3.85	5.19
ResnetRegressor	4.54	6.04	4.58	4.54	6.31
FTTransformerRegressor	5.31	4.54	5.54	5.27	5.27
Friedman χ^2	114.3	96.0	93.0	135.4	104.2
Significance	*	*	*	*	*
<i>Significance codes: * $p \lll 0.001$</i>					

Conclusion

- ANNs yet to completely outperform tree-based models in OOD extrapolation.
- Almost DL models are top models which may be indicative of prospect of robust DL models under OOD assessment.
- Side experiment supports the definition of interpolation and extrapolation using convex notions.

- Computational resources limitations
- Hyperparameters tuning
- Lack of feature engineering and domain knowledge

References I

- [1] I. Gulrajani and D. Lopez-Paz, *In search of lost domain generalization*, 2020. arXiv: 2007.01434 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2007.01434>.
- [2] L. Tamang, M. R. Bouadjenek, R. Dazeley, and S. Aryal, *Handling out-of-distribution data: A survey*, 2025. arXiv: 2507.21160 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2507.21160>.
- [3] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012, ISSN: 0031-3203. DOI: <https://doi.org/10.1016/j.patcog.2011.06.019>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320311002901>.

- [4] R. Balestriero, J. Pesenti, and Y. LeCun, *Learning in high dimension always amounts to extrapolation*, 2021. arXiv: 2110.09485 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2110.09485>.
- [5] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge: Cambridge University Press, 2004, ISBN: 9780521833783.